COMICCRAFT — CODE LAYOUT, READABILITY AND REUSABILITY

==========

FOLDER STRUCTURE

```
ComicCraft/
├── app/
│   ├── __init__.py          (empty)
│   ├── main.py              (routes and orchestration)
│   ├── gemini_outline.py    (5-panel outline via Gemini)
│   ├── gemini_story.py      (narration via Gemini)
│   └── image_generator.py   (images via Pollinations)
├── templates/
│   ├── index.html
│   └── comic_preview.html
└── static/
    ├── style.css
    └── panels/
```

SEPARATION OF CONCERNS

gemini_outline.py : Only creates the panel outline

gemini_story.py : Only writes narration and dialogue

image_generator.py : Only creates images

main.py : Only handles HTTP routes and layout assembly

Each module can be tested independently.

READABILITY PRACTICES

- Descriptive function names (generate_outline, split_story_by_panel)
- Docstrings on every function
- Constants at the top (MODEL_NAME, POLLINATIONS_URL)
- No magic numbers (768x768 image size defined as parameters)
- Clear variable names (full_prompt, story_chunks, layout)

REUSABILITY

- image_generator.py works in any project that needs AI images
- generate_outline() can be reused in any storytelling app
- split_story_by_panel() is a pure function — no side effects
- The save_pdf() function from exporters.py is portable

NAMING CONVENTIONS

Files : snake_case (gemini_outline.py)
Classes : PascalCase (ComicPDF)
Functions : snake_case (generate_outline)
Constants : UPPER_CASE (MODEL_NAME, POLLINATIONS_URL)
Templates : lowercase with underscores (comic_preview.html)